When to Deploy a Model: Deployment Readiness Checklist

Interview Answer Framework

Short Answer:
"A model is ready for deployment when it meets four key criteria: (1) Performance benchmarks on test data, (2) Business requirements and ROI targets, (3) Safety and reliability checks, and (4) Operational readiness including monitoring and rollback plans."

Detailed Answer:

1. Performance Criteria ■

A. Benchmark Thresholds

Define minimum acceptable performance:

```
DEPLOYMENT_CRITERIA = {
    'accuracy': 0.85,           # 85% minimum
    'precision': 0.80,          # 80% minimum
    'recall': 0.75,             # 75% minimum
    'f1_score': 0.80,           # 80% minimum
    'latency_p99': 500,         # 500ms max
    'throughput': 100           # 100 req/sec min
}

def is_deployment_ready(model_metrics):
    """Check if model meets deployment criteria"""
    for metric, threshold in DEPLOYMENT_CRITERIA.items():
        if model_metrics[metric] < threshold:
            print(f"■ {metric}: {model_metrics[metric]} < {threshold}")
            return False
    return True

# Example
model_metrics = {
    'accuracy': 0.87,
    'precision': 0.85,
    'recall': 0.82,
    'f1_score': 0.83,
    'latency_p99': 450,
    'throughput': 120
}

ready = is_deployment_ready(model_metrics)
print(f"Deployment ready: {ready}")  # True
```

B. Beats Baseline

Model must outperform current system:

```
class BaselineComparison:
    def __init__(self, baseline_performance):
        self.baseline = baseline_performance

    def check_improvement(self, new_model_performance, min_improvement=0.05):
        """New model must be at least 5% better"""

        improvement = {}
        meets_threshold = True

        for metric, baseline_value in self.baseline.items():
            new_value = new_model_performance.get(metric, 0)
            improvement[metric] = new_value - baseline_value

            if improvement[metric] < min_improvement:
                print(f"■ {metric}: +{improvement[metric]:.3f} < +{min_improvement}")
                meets_threshold = False
            else:
                print(f"■ {metric}: +{improvement[metric]:.3f}")

        return meets_threshold

# Example
baseline = {
    'accuracy': 0.75,
    'f1_score': 0.72
}

new_model = {
    'accuracy': 0.87,  # +12% improvement ■
    'f1_score': 0.83   # +11% improvement ■
}

comparator = BaselineComparison(baseline)
can_deploy = comparator.check_improvement(new_model)
```

Key point: New model must provide **meaningful improvement** over existing system.

C. Performance on Edge Cases

```
def test_edge_cases(model, edge_cases):
    """Test model on difficult/rare cases"""

    results = {
        'overall_accuracy': 0,
        'edge_case_accuracy': 0,
        'failure_modes': []
    }

    for case in edge_cases:
        prediction = model.predict(case['input'])
        correct = prediction == case['expected']

        if not correct:
            results['failure_modes'].append({
                'input': case['input'],
                'expected': case['expected'],
                'predicted': prediction,
                'case_type': case['type']
            })

    results['edge_case_accuracy'] = (
        len([c for c in edge_cases if model.predict(c['input']) == c['expected']]) /
        len(edge_cases)
    )

    # Must handle at least 70% of edge cases
    if results['edge_case_accuracy'] < 0.70:
```

```
        print(f"■ Edge case accuracy: {results['edge_case_accuracy']:.2%}")
        return False

    return True

# Example edge cases
edge_cases = [
    {'input': 'empty string', 'expected': 'NO_INPUT', 'type': 'empty'},
    {'input': 'very long text...', 'expected': 'TRUNCATED', 'type': 'long'},
    {'input': 'unicode™©®', 'expected': 'UNICODE', 'type': 'special_chars'}
]
```

2. Business Requirements ■

A. Business Metrics Met

```
class BusinessMetrics:
    """Evaluate business impact, not just model metrics"""

    def __init__(self, requirements):
        self.requirements = requirements

    def calculate_roi(self, model_performance):
        """Calculate return on investment"""

        # Cost savings from automation
        automated_tasks = model_performance['throughput'] * 24 * 30
        cost_per_manual_task = 0.50  # $0.50 per manual task
        monthly_savings = automated_tasks * cost_per_manual_task

        # Model costs
        api_calls = automated_tasks
        cost_per_call = 0.001  # $0.001 per API call
        monthly_cost = api_calls * cost_per_call

        # ROI
        net_savings = monthly_savings - monthly_cost
        roi = net_savings / monthly_cost if monthly_cost > 0 else 0

        return {
            'monthly_savings': monthly_savings,
            'monthly_cost': monthly_cost,
            'net_savings': net_savings,
            'roi': roi,
            'roi_percentage': roi * 100
        }

    def meets_business_requirements(self, model_performance):
        """Check if business requirements are met"""

        roi = self.calculate_roi(model_performance)

        checks = {
            'roi': roi['roi_percentage'] >= self.requirements['min_roi_percent'],
            'cost': roi['monthly_cost'] <= self.requirements['max_monthly_cost'],
            'accuracy': model_performance['accuracy'] >= self.requirements['min_accuracy']
        }

        return all(checks.values()), checks, roi

# Example
requirements = {
    'min_roi_percent': 200,      # 200% ROI minimum
    'max_monthly_cost': 10000,   # $10K max
    'min_accuracy': 0.85         # 85% accuracy
}

business = BusinessMetrics(requirements)
```

```
ready, checks, roi = business.meets_business_requirements({
    'accuracy': 0.87,
    'throughput': 100
})

print(f"Business ready: {ready}")
print(f"ROI: {roi['roi_percentage']:.1f}%")
```

Key questions:
- Does it save money?
- Does it improve user experience?
- Does it generate revenue?

B. User Experience Threshold

```
def user_experience_check(model_metrics):
    """Check if UX is acceptable"""

    checks = {
        'latency': model_metrics['latency_p99'] < 1000,  # < 1 second
        'accuracy': model_metrics['accuracy'] > 0.90,    # > 90%
        'availability': model_metrics['uptime'] > 0.99   # 99% uptime
    }

    # All must pass for good UX
    if not all(checks.values()):
        print("■ UX requirements not met:")
        for check, passed in checks.items():
            print(f"  {check}: {'■' if passed else '■'}")
        return False

    return True
```

3. Safety & Reliability ■

A. Error Rate Acceptable

```
class SafetyChecks:
    """Ensure model is safe to deploy"""

    def __init__(self, max_error_rate=0.05):
        self.max_error_rate = max_error_rate

    def check_error_distribution(self, predictions, labels):
        """Check error patterns"""

        errors = predictions != labels
        error_rate = errors.sum() / len(errors)

        if error_rate > self.max_error_rate:
            print(f"■ Error rate: {error_rate:.2%} > {self.max_error_rate:.2%}")
            return False

        # Check for systematic errors (bias)
        error_distribution = self.analyze_error_distribution(predictions, labels, errors)

        if error_distribution['max_group_error'] > 2 * error_rate:
            print(f"■ Systematic bias detected")
            return False
```

```
        return True

    def analyze_error_distribution(self, predictions, labels, errors):
        """Analyze if errors are uniformly distributed"""
        # Implementation depends on data structure
        return {'max_group_error': 0.08}

    def catastrophic_failure_test(self, model, test_cases):
        """Ensure no catastrophic failures"""

        catastrophic_failures = []

        for case in test_cases:
            try:
                prediction = model.predict(case['input'])

                # Check for unacceptable outputs
                if self.is_catastrophic(prediction, case):
                    catastrophic_failures.append({
                        'input': case['input'],
                        'output': prediction,
                        'reason': 'Unacceptable output'
                    })
            except Exception as e:
                catastrophic_failures.append({
                    'input': case['input'],
                    'error': str(e),
                    'reason': 'Exception'
                })

        if len(catastrophic_failures) > 0:
            print(f"■ {len(catastrophic_failures)} catastrophic failures")
            return False

        return True

    def is_catastrophic(self, prediction, case):
        """Define what counts as catastrophic failure"""
        # Examples:
        # - Toxic/harmful content
        # - Completely wrong answer in safety-critical domain
        # - Leaking PII
        return False  # Implement based on use case
```

B. Bias & Fairness Check

```
from sklearn.metrics import confusion_matrix
import numpy as np

def fairness_check(model, test_data, protected_attribute):
    """Check for bias across protected groups"""

    groups = test_data[protected_attribute].unique()
    metrics_by_group = {}

    for group in groups:
        group_data = test_data[test_data[protected_attribute] == group]
        predictions = model.predict(group_data['X'])

        metrics_by_group[group] = {
            'accuracy': (predictions == group_data['y']).mean(),
            'size': len(group_data)
        }

    # Check for disparate impact
    accuracies = [m['accuracy'] for m in metrics_by_group.values()]
    max_accuracy = max(accuracies)
    min_accuracy = min(accuracies)

    # 80% rule: lowest accuracy should be at least 80% of highest
    disparate_impact_ratio = min_accuracy / max_accuracy
```

```
    if disparate_impact_ratio < 0.80:
        print(f"■ Fairness issue: {disparate_impact_ratio:.2%} < 80%")
        print("Metrics by group:")
        for group, metrics in metrics_by_group.items():
            print(f"  {group}: {metrics['accuracy']:.2%}")
        return False

    return True
```

C. Adversarial Robustness

```
def adversarial_test(model, test_cases):
    """Test against adversarial inputs"""

    adversarial_cases = [
        {'input': 'Normal input', 'expected': 'NORMAL'},
        {'input': 'Adversarial™input®', 'expected': 'ADVERSARIAL'},
        {'input': 'Input with\nnewlines\nand\ttabs', 'expected': 'NORMAL'},
        {'input': '', 'expected': 'EMPTY'},
        {'input': 'Very ' + 'long ' * 1000, 'expected': 'LONG'}
    ]

    robust = True

    for case in adversarial_cases:
        try:
            prediction = model.predict(case['input'])
            if prediction != case['expected']:
                print(f"■ Failed adversarial case: {case['input'][:50]}")
                robust = False
        except Exception as e:
            print(f"■ Exception on adversarial case: {e}")
            robust = False

    return robust
```

4. Operational Readiness ■

A. Monitoring in Place

```
class MonitoringSetup:
    """Ensure monitoring is ready before deployment"""

    def __init__(self):
        self.required_monitors = [
            'latency_p99',
            'throughput',
            'error_rate',
            'model_drift',
            'data_drift',
            'feature_distribution'
        ]

    def verify_monitoring(self):
        """Check if all monitors are configured"""

        configured = []
        missing = []

        for monitor in self.required_monitors:
            if self.is_monitor_configured(monitor):
                configured.append(monitor)
```

```
            else:
                missing.append(monitor)

        if missing:
            print(f"■ Missing monitors: {missing}")
            return False

        print(f"■ All {len(configured)} monitors configured")
        return True

    def is_monitor_configured(self, monitor):
        """Check if specific monitor exists"""
        # Implementation would check actual monitoring system
        return True

# Example
monitoring = MonitoringSetup()
ready = monitoring.verify_monitoring()
```

Required monitoring:
- Performance metrics (latency, throughput)
- Model quality (accuracy, f1)
- Data drift detection
- Feature distribution tracking
- Error rates by category
- Resource usage (CPU, memory)

B. Rollback Plan Exists

```
class DeploymentStrategy:
    """Deployment strategy with rollback capability"""

    def __init__(self):
        self.deployment_stages = [
            'canary',      # 5% traffic
            'staged',      # 50% traffic
            'full'         # 100% traffic
        ]

    def can_proceed_to_next_stage(self, current_stage, metrics):
        """Decide if safe to proceed"""

        # Define success criteria for each stage
        criteria = {
            'canary': {
                'error_rate': 0.05,
                'latency_p99': 1000,
                'min_samples': 1000
            },
            'staged': {
                'error_rate': 0.03,
                'latency_p99': 800,
                'min_samples': 10000
            }
        }

        if current_stage not in criteria:
            return True

        stage_criteria = criteria[current_stage]

        checks = {
            'error_rate': metrics['error_rate'] <= stage_criteria['error_rate'],
            'latency': metrics['latency_p99'] <= stage_criteria['latency_p99'],
            'samples': metrics['total_requests'] >= stage_criteria['min_samples']
        }

        if not all(checks.values()):
```

```
            print(f"■ {current_stage} stage criteria not met:")
            for check, passed in checks.items():
                print(f"  {check}: {'■' if passed else '■'}")
            return False

        return True

    def should_rollback(self, metrics, baseline_metrics):
        """Decide if we should rollback"""

        # Rollback if significant degradation
        if metrics['error_rate'] > baseline_metrics['error_rate'] * 1.5:
            print("■ Error rate increased by 50%, rolling back")
            return True

        if metrics['latency_p99'] > baseline_metrics['latency_p99'] * 1.3:
            print("■ Latency increased by 30%, rolling back")
            return True

        return False
```

C. A/B Testing Configured

```
class ABTestSetup:
    """A/B test configuration"""

    def __init__(self, control_model, treatment_model):
        self.control = control_model
        self.treatment = treatment_model
        self.traffic_split = 0.50  # 50/50 split

    def calculate_sample_size(self, baseline_rate, mde, alpha=0.05, power=0.80):
        """Calculate required sample size for A/B test"""
        from scipy.stats import norm

        # Minimum Detectable Effect (MDE)
        # e.g., detect 5% improvement

        z_alpha = norm.ppf(1 - alpha/2)
        z_beta = norm.ppf(power)

        p1 = baseline_rate
        p2 = baseline_rate * (1 + mde)

        n = (
            (z_alpha + z_beta) ** 2 *
            (p1 * (1-p1) + p2 * (1-p2))
        ) / (p1 - p2) ** 2

        return int(n * 2)  # Total for both groups

    def is_significant(self, control_metrics, treatment_metrics, alpha=0.05):
        """Check if treatment is significantly better"""
        from scipy.stats import ttest_ind

        t_stat, p_value = ttest_ind(
            control_metrics['accuracies'],
            treatment_metrics['accuracies']
        )

        if p_value < alpha and treatment_metrics['mean_accuracy'] > control_metrics['mean_accu
            print(f"■ Treatment significantly better (p={p_value:.4f})")
            return True

        print(f"■ Not significant or worse (p={p_value:.4f})")
        return False

# Example
ab_test = ABTestSetup(control_model=None, treatment_model=None)
n = ab_test.calculate_sample_size(
    baseline_rate=0.75,
```

```
    mde=0.05  # Detect 5% improvement
)
print(f"Need {n:,} samples for A/B test")
```

Complete Deployment Checklist

```
class DeploymentReadinessChecker:
    """Complete pre-deployment checklist"""

    def __init__(self, model, test_data, business_requirements):
        self.model = model
        self.test_data = test_data
        self.requirements = business_requirements
        self.checks = {}

    def run_all_checks(self):
        """Run complete deployment readiness check"""

        print("="*60)
        print("DEPLOYMENT READINESS CHECKLIST")
        print("="*60)

        # 1. Performance
        print("\n1. PERFORMANCE CHECKS")
        self.checks['performance'] = self.check_performance()

        # 2. Business
        print("\n2. BUSINESS REQUIREMENTS")
        self.checks['business'] = self.check_business_requirements()

        # 3. Safety
        print("\n3. SAFETY & RELIABILITY")
        self.checks['safety'] = self.check_safety()

        # 4. Operational
        print("\n4. OPERATIONAL READINESS")
        self.checks['operational'] = self.check_operational()

        # Final decision
        all_passed = all(self.checks.values())

        print("\n" + "="*60)
        print("SUMMARY")
        print("="*60)
        for category, passed in self.checks.items():
            status = "■ PASS" if passed else "■ FAIL"
            print(f"{category.upper()}: {status}")

        print("\n" + "="*60)
        if all_passed:
            print("■ MODEL IS READY FOR DEPLOYMENT")
        else:
            print("■ MODEL NOT READY - FIX ISSUES FIRST")
        print("="*60)

        return all_passed

    def check_performance(self):
        """Performance checks"""
        metrics = self.evaluate_model()

        checks = [
            metrics['accuracy'] >= 0.85,
            metrics['latency_p99'] <= 500,
            metrics['throughput'] >= 100
        ]

        return all(checks)

    def check_business_requirements(self):
        """Business requirement checks"""
```

```
        roi = self.calculate_roi()
        return roi['roi_percentage'] >= 200

    def check_safety(self):
        """Safety checks"""
        return (
            self.check_error_rate() and
            self.check_fairness() and
            self.check_adversarial_robustness()
        )

    def check_operational(self):
        """Operational readiness checks"""
        return (
            self.verify_monitoring() and
            self.verify_rollback_plan() and
            self.verify_ab_test_setup()
        )

    # Helper methods (simplified)
    def evaluate_model(self):
        return {'accuracy': 0.87, 'latency_p99': 450, 'throughput': 120}

    def calculate_roi(self):
        return {'roi_percentage': 250}

    def check_error_rate(self):
        return True

    def check_fairness(self):
        return True

    def check_adversarial_robustness(self):
        return True

    def verify_monitoring(self):
        return True

    def verify_rollback_plan(self):
        return True

    def verify_ab_test_setup(self):
        return True

# Usage
checker = DeploymentReadinessChecker(
    model=my_model,
    test_data=test_dataset,
    business_requirements=requirements
)

ready_to_deploy = checker.run_all_checks()
```

Output:

```
============================================================
DEPLOYMENT READINESS CHECKLIST
============================================================

1. PERFORMANCE CHECKS
■ Accuracy: 0.87 >= 0.85
■ Latency: 450ms <= 500ms
■ Throughput: 120 >= 100 req/sec

2. BUSINESS REQUIREMENTS
■ ROI: 250% >= 200%
■ Monthly cost: $8,500 <= $10,000
■ User satisfaction: 4.2/5 >= 4.0/5

3. SAFETY & RELIABILITY
■ Error rate: 3.2% <= 5%
■ Fairness check passed
■ Adversarial robustness passed
```

```
4. OPERATIONAL READINESS
■ Monitoring configured
■ Rollback plan documented
■ A/B test ready

============================================================
SUMMARY
============================================================
PERFORMANCE: ■ PASS
BUSINESS: ■ PASS
SAFETY: ■ PASS
OPERATIONAL: ■ PASS

============================================================
■ MODEL IS READY FOR DEPLOYMENT
============================================================
```

Interview Answer (Complete)

Question: "When do you decide a model can be deployed?"

Answer:

"I use a four-pillar framework to determine deployment readiness:

1. Performance (Technical)
- Meets accuracy threshold (e.g., >85%)
- Beats baseline by meaningful margin (>5%)
- Acceptable latency (p99 < 500ms)
- Handles edge cases (>70% accuracy)

2. Business Value
- Positive ROI (typically >200%)
- Meets user experience requirements
- Aligns with business goals
- Cost is acceptable

3. Safety & Reliability
- Error rate below threshold (<5%)
- No systematic bias (fairness checks pass)
- Robust to adversarial inputs
- No catastrophic failure modes

4. Operational Readiness
- Monitoring in place (metrics, alerts)
- Rollback plan documented
- A/B testing configured
- Team trained on new model

I implement this as an automated checklist that must pass before any deployment. We typically do a gradual rollout: 5% canary → 50% staged → 100% full, with automatic rollback if metrics degrade.

For high-risk systems, we add an additional human review step and require sign-off from stakeholders."

Red Flags (Do NOT Deploy If...)

■ Model not tested on production-like data
■ No monitoring configured
■ No rollback plan
■ Significant bias detected
■ ROI is negative
■ Team hasn't been trained
■ Error rate is higher than current system
■ Catastrophic failure possible
■ No A/B test planned
■ Business stakeholders not aligned

Summary Checklist

```
■ Performance benchmarks met
■ Beats baseline by >5%
■ Edge cases handled
■ Positive ROI
■ User experience acceptable
■ Error rate < 5%
■ Fairness checks passed
■ Adversarial robustness verified
■ Monitoring configured
■ Rollback plan ready
■ A/B test setup complete
■ Team trained
■ Stakeholders aligned
```

Only deploy when ALL checks pass! ■